

# Review

## Chapter 16 Recommended Approach



UNIVERSITY  
OF  
JOHANNESBURG

# Reviews

## What are they?

- A meeting conducted by technical people.
- A technical assessment of a work product created during the software engineering process.
- A software quality assurance mechanism.
- A training ground.

## What they are not!

- A project summary or progress assessment.
  - A meeting intended solely to impart information.
  - A mechanism for political or personal revenge!
- 



# Cost Impact of Software Defects

- *Error*—a quality problem found *before* the software is released to end users.
- *Defect*—a quality problem found only *after* the software has been released to end-users.
- We make this distinction because errors and defects have very different economic, business, psychological, and human impact.



# Defect Amplification and Removal

- **Defect amplification** is a term used to describe how a defect introduced early in the software engineering workflow (for example: during requirement modeling) and undetected, can and often will be **amplified into multiple errors** during design and more errors in construction.
- **Defect propagation** is a term used to **describe the impact** an undiscovered defect has on future development activities or product behavior.
- **Technical debt** is the term used to describe the **costs incurred by failing to find and fix defects early** or failing to update documentation following software changes.





# Informal Reviews

- The obvious benefits of technical reviews are the early discovery of errors so that they do not propagate to the next step in the software process.
- Many different types of reviews can be conducted as part of software engineering. Each has its place.
- An informal meeting around the coffee machine is a form of review, if technical problems are discussed.



# Formal Reviews

- A formal presentation of software architecture to an audience or customers, management, and technical staff is also a form of review.
- Within the context of the software process, the terms *defects* and *fault* are synonymous.
- Both imply a quality problem that is discovered after the software has been released to end users.



# Cost Impact of Software Defects

- Several industry studies indicated that design activities introduce between 50-65 % of all errors during the software process.
- However, review techniques have been shown to be up to 75% effective in uncovering design errors.
- The review process substantially reduces the cost of subsequent activities in the software process.
- The sooner you find a defect the cheaper it is to fix it.



# Review Metrics and their use

- Technical reviews are one of many actions that are required as part of good software engineering practice.
  - Each action requires human effort.
  - Since available project effort is finite, it is important for a software engineering organization to understand the effectiveness of each action by defining a set of metrics.
  - Although many metrics can be defined for technical reviews, a relatively small subset can provide useful insight.
  - The following review metrics can be used for each review that is conducted.
- 





# Review metrics

- **Preparation effort, ( $E_p$ )** – the effort (in person hours) required to review a **work product** prior to the actual review meeting.
- **Assessment effort, ( $E_a$ )** – the effort (in person hours) that is expended during the actual review.
- **Rework effort, ( $E_r$ )** – the effort (in person hours) that is dedicated to the correction of these errors uncovered during the review
- **Work Product size, WPS** – a measure of the size of the work product that has been reviewed (eg. Number of UML models, or number of document pages, or the number of lines of code)
- **Minor errors found,  $Err_{minor}$**  – the number of errors found that can be categorized as minor (requiring less than some prespecified effort to correct)
- **Major errors found,  $Err_{major}$**  – the number of errors found that can be categorized as major (requiring more than some prespecified effort to correct)



# Review metrics

## Review effort metrics

Review effort ( $E_{\text{review}}$ ) = Preparation effort, ( $E_p$ ) + Assessment effort, ( $E_a$ ) + Rework effort, ( $E_r$ )

## Total error metrics

$$Err_{\text{tot}} = Err_{\text{minor}} + Err_{\text{major}}$$

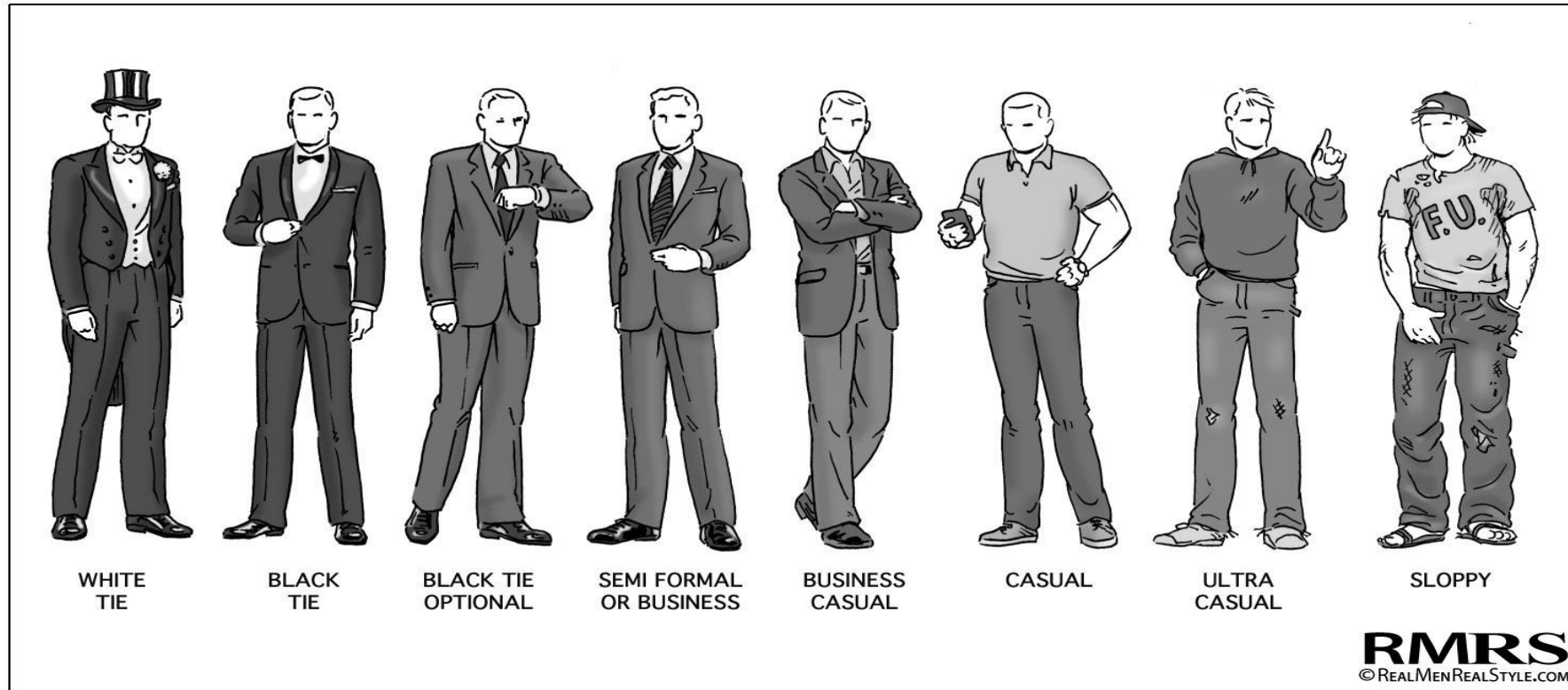
## Error density metrics

$$\text{Error Density} = \frac{Err_{\text{tot}}}{WPS \text{ (work Product size)}}$$



# Review metrics and their use

Technical reviews should be applied with a level of formality that is appropriate for the product to be built, the project timeline, and the people who are doing the work.





# Formal Technical Review



# Review guidelines for a formal review

1. Review the product, not the producer.
2. Set an agenda and maintain it.
3. Limit debate and rebuttal.
4. Voice problem areas, but don't attempt to solve every problem noted.
5. Take written notes.





# Review guidelines for a formal review

6. Limit the number of participants and insist upon advance preparation.
7. Develop a checklist for each product that is likely to be reviewed.
  - Analysis
  - Design
  - Code
  - Testing
8. Allocate resources and schedule time for Formal Technical Reviews.
9. Conduct meaningful training for all reviewers.
10. Review your early reviews.



# Postmortem Evaluation

- Lessons learned after the product has been delivered to the customer
- Analyse the accuracy of the review metrics used

